

BTP610 Test 4 / Final Assessment Reference Sheet

Firebase | Firestore | Hooks | Expo Router | Camera

Firestore Setup

- `initializeApp(config)`: Firebase app.
- `getAuth(app)`: Authentication service.
- `getFirestore(app)`: Firestore service.
- `firebaseConfig.ts`: project connection settings; exports shared instances for reuse.
- Auth: credentials. Firestore: app data.

Authentication API

- `createUserWithEmailAndPassword`: create account; arguments are auth, email, password; returns `UserCredential`.
- `signInWithEmailAndPassword`: sign in existing account; same arguments/return type.
- `signOut(auth)`: sign out current user.
- Credential values: `user.uid` (unique Auth ID), `user.email`.
- Await auth calls inside `try/catch`.

Auth State

- `undefined`: still checking; do not route.
- `null`: check finished; logged out.
- `User`: logged-in user object.
- Type: `User | null | undefined`.
- `onAuthStateChanged`: listens for sign-in/out; returns unsubscribe cleanup.

Sign-Up Flow

Validate → create Auth account → read UID → save non-password profile in Firestore at UID → route.

Firestore Model / IDs

- Structure: Collection → Document → Fields.
- `collection(db, name)`: collection reference, not data/array.
- `doc(db, collection, id)`: one document reference.
- Snapshot: `doc.id = ID`; `doc.data()` = fields.
- BookDB: auto IDs for independent records.
- `userProfile`: Auth UID as known ID for direct lookup.
- Store `uid` in user-owned records to link data to its owner.

Firestore CRUD

- `addDoc(collectionRef, data)`: create with auto ID; returns document reference.
- `setDoc(documentRef, data)`: write at known ID; replaces by default.
- `getDoc(documentRef)`: one-time document read.
- `getDocs(collectionRef)`: one-time collection read.
- `updateDoc(documentRef, fields)`: change selected fields; preserve others.
- `deleteDoc(documentRef)`: permanent delete.

nent delete.

Real-Time vs. One-Time

- `getDoc/getDocs`: read once; later changes do not update UI.
- `onSnapshot(ref, callback)`: current data, then callback after each matching add/edit/delete.
- `snapshot.docs`: current matching documents.
- List: map snapshots to `id + data()`, then set state.
- Flow: DB change → callback → snapshot → state → UI.
- It returns unsubscribe; use as `useEffect` cleanup to avoid leaks/duplicates.

Hooks

- `useState`: value affects UI; setter causes re-render. Never mutate state directly.
- `useRef`: remembers value/component through `.current`; change does not re-render.
- State: form/errors/list/camera facing. Ref: camera component/readiness.
- `useEffect`: side effects after render.
- No dependency array: every render. Empty `[]`: mount once. Listed dependencies: mount + when changed.
- Returned function: cleanup before re-run and on unmount.

Object State / Custom Hook

- Field update: spread existing object, then override target field; no spread loses other fields.
- `!!value`: Boolean conversion; empty string false, non-empty true.
- Custom hook begins with `use`; reuses hook/listener logic. Auth hook listens, stores user, cleans up, returns user.

Expo Router

- Files/folders under `app/` map to routes; `index.tsx` is directory default.
- `_layout.tsx`: Stack/Tabs configuration and wrapper.
- `(auth)/(welcome)/(tabs)`: organization only; no URL segment.
- `Redirect`: route decision during render.
- `navigate/push`: add route; Back can return.
- `replace`: replace route; Back cannot return. Use after login/logout.

Auth Routing Decision

Open → check auth: `undefined` → wait; `null` → sign-in; `User` → app. Early routing can flash wrong screen.

Stacks, Tabs, Headers

- `Stack`: history/back. `Tabs`: persistent sibling navigation.
- `Tabs.Screen`: title, icon, header, tab options.
- `tabBarActiveTintColor`: active icon/label color.

- `headerRight`: header button/icon; `headerShown`: show/hide header.

FlatList / Components

- Props: `data`, `renderItem`, `keyExtractor`.
- Stable unique key required; Firestore document ID works.
- `renderItem` receives `item`; reusable item component improves clarity.

Forms / TextInput

- Controlled input: `value` from state; `onChangeText` updates state.
- Validate before async call; return early if invalid.
- `secureTextInput`: hide password. `keyboardType`: email/phone keyboard. `autoCapitalize`: `none/words`.
- Non-empty error string is truthy; use for conditional display.

Camera / Permissions

- Packages: `expo-camera`, `expo-media-library`.
- `CameraView`: `preview`; `CameraType`: `front/back`.
- `useCameraPermissions()`: camera status + request function.
- `MediaLibrary.usePermissions()`: separate gallery status + request.
- Camera permission permits capture; media permission permits saving. One does not grant the other.

Camera Flow / API

1. Request both permissions; render camera only if granted.
2. Attach `useRef<CameraView>`; track `onCameraReady`.
3. Guard action: mounted, ready, ref not null.
4. `takePictureAsync()`: capture; result has `uri`.
5. `createAssetAsync(uri)`: save to gallery.
6. Change facing state to toggle front/back.

Expo Go Limitation

Album browse/create/add APIs need full media access and may require a dedicated development build, not Expo Go.

End-to-End Flows

- Sign in: auth call → state change → listener gets `User` → app route.
- CRUD: action → Firestore → snapshot → state → UI.
- Sign out → user null → sign-in route.

Common Traps

- Reference $\neq$ data; `undefined` $\neq$ logged out.
- Listener needs cleanup; route group $\neq$ URL segment.
- Ref change $\neq$ re-render; camera permission $\neq$ media permission.
- Guard camera permission/readiness; preserve object fields with spread.